

Week 15 Homework: Train a Tiny Neural Network by Hand

Goal: finish the MLP and classifier-evaluation material, derive backpropagation, implement one training loop, and explain GPU acceleration and memory use.

Deliverables. Submit (1) written calculations, (2) one Python file named `week15_tiny_mlp.py`, (3) one loss-curve figure, (4) one CPU/GPU and inference/training memory diagram, and (5) the Feynman Technique post.

AI policy. You may use AI or web search to understand official NumPy or PyTorch documentation, ask conceptual questions after first attempting a problem, or interpret an error message. You may **not** ask AI to derive the required gradients, fill the TODO code, calculate the confusion matrix, write the memory explanation, or produce the Feynman post. Complete the core work manually. If you use AI, add one sentence stating exactly what it helped you understand.

Rules for the core implementation. Use NumPy only. Do not use PyTorch, TensorFlow, JAX, scikit-learn neural-network models, automatic differentiation, or an AI-written implementation. Matrix multiplication with `@` is allowed. You may use `sklearn.metrics` only at the very end to check your manually implemented metrics.

Monday: Recover the MLP Forward Pass and Classifier Evaluation

For a binary classifier, suppose

$$y_{\text{true}} = [1, 0, 1, 1, 0, 0, 1, 0, 1, 0]$$

and

$$\hat{p} = [.91, .72, .64, .42, .38, .81, .55, .20, .49, .10].$$

1. At threshold $t = .50$, write the predicted-label vector.
2. Count TP, TN, FP, and FN. Mark every example with its category.
3. Calculate accuracy, precision, recall, and F1-score by hand.
4. Repeat the predicted labels and four counts at $t = .70$.
5. Which threshold has higher precision? Which has higher recall? Explain the change without saying that one threshold is always better.
6. If a false positive means paying for an unnecessary field experiment, which threshold might be preferred? If a false negative means missing a potentially valuable safety message, how might the preference change?
7. Explain why these test-set metrics are not the same thing as the loss used to update one network weight.

Add and complete the following functions in `week15_tiny_mlp.py`.

```
import numpy as np

def classify(probabilities, threshold=0.5):
    # TODO: return an integer array containing 0 or 1
    pass

def confusion_counts(y_true, y_pred):
    tp = tn = fp = fn = 0
    for truth, prediction in zip(y_true, y_pred):
        # TODO: update exactly one count
        pass
    return tp, tn, fp, fn

def classification_metrics(tp, tn, fp, fn):
    # TODO: calculate accuracy, precision, recall, and F1
    # Handle a zero denominator by returning 0 for that metric.
    pass
```

Confirm that the code exactly matches your hand calculations at $t = .50$ and $t = .70$. Then make a table for thresholds .30, .40, .50, .60, .70, and .80 containing TP, FP, FN, precision, recall, and F1.

Tuesday: One Complete Forward Pass and Loss

Use this network and one training example:

$$x = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad y = 1,$$

$$W^{(1)} = \begin{bmatrix} .5 & -.4 \\ .3 & .2 \end{bmatrix}, \quad b^{(1)} = \begin{bmatrix} .1 \\ -.1 \end{bmatrix}, \quad W^{(2)} = \begin{bmatrix} -.7 & 1.2 \end{bmatrix}, \quad b^{(2)} = -.2.$$

The hidden activation is ReLU and the output activation is sigmoid:

$$z^{(1)} = W^{(1)}x + b^{(1)}, \quad h = \text{ReLU}(z^{(1)}),$$

$$z^{(2)} = W^{(2)}h + b^{(2)}, \quad \hat{p} = \sigma(z^{(2)}).$$

1. Calculate both hidden scores $z_1^{(1)}, z_2^{(1)}$.
2. Apply ReLU to obtain h_1, h_2 .
3. Calculate $z^{(2)}$ and $\hat{p}$, rounding the probability to three decimals.
4. Calculate binary cross-entropy

$$L = -[y \ln \hat{p} + (1 - y) \ln(1 - \hat{p})].$$

5. Draw the two-input, two-hidden-neuron, one-output network. Label every edge with its weight and every neuron with its score and activation.
6. Explain why the output probability and the thresholded class are both useful but serve different purposes.

Complete these functions and confirm that the printed values match your calculations.

```
def relu(z):
    # TODO
    pass

def sigmoid(z):
    # TODO: use np.exp
    pass

def binary_cross_entropy(y, p):
    # TODO: clip p to [1e-12, 1 - 1e-12], then calculate mean loss
    pass

def forward(X, W1, b1, W2, b2):
    # TODO: z1 = ...
    # TODO: h = ...
    # TODO: z2 = ...
    # TODO: p = ...
    # Return every value needed later by backpropagation.
    pass
```

Use two-dimensional arrays for batches: `X.shape == (batch, 2)`, `W1.shape == (hidden, 2)`, and `W2.shape == (1, hidden)`.

Wednesday: Derive Backpropagation Manually

Use the same example and the values from Tuesday. Show every substitution, sign, and matrix shape.

1. For sigmoid plus binary cross-entropy, calculate

$$\delta^{(2)} = \frac{\partial L}{\partial z^{(2)}} = \hat{p} - y.$$

Explain what its sign says about the direction in which $z^{(2)}$ should move.

2. Derive and calculate

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} h^T, \quad \frac{\partial L}{\partial b^{(2)}} = \delta^{(2)}.$$

3. Calculate the error signal arriving at the hidden activations:

$$(W^{(2)})^T \delta^{(2)}.$$

4. Calculate $\text{ReLU}'(z^{(1)})$ for both hidden neurons. Explain why one neuron receives zero gradient from this example.

5. Calculate

$$\delta^{(1)} = \left((W^{(2)})^T \delta^{(2)} \right) \odot \text{ReLU}'(z^{(1)}).$$

6. Derive and calculate

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} x^T, \quad \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)}.$$

7. Check that each gradient has the same shape as the parameter it updates.

8. With learning rate $\eta = .1$, calculate all updated weights and biases using

$$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial L}{\partial \theta}.$$

9. Perform a new forward pass. Report the new probability and loss. Did this step improve this example?

Required chain-rule explanation. In four to six sentences, explain backpropagation without using the phrases “the computer just knows” or “the error goes backward.” State what is actually propagated and why stored forward-pass values are needed.

Thursday: Implement and Check the Backward Pass

Complete the function below. The division by batch size makes the gradients correspond to the *mean* batch loss.

```
def backward(X, y, W2, z1, h, p):
    batch_size = X.shape[0]

    # Output layer: sigmoid + binary cross-entropy
    # TODO: delta2 = (p - y) / batch_size
    # TODO: dW2 = ...
    # TODO: db2 = ... with shape (1,)

    # Hidden layer
    # TODO: relu_derivative = ...
    # TODO: delta1 = ...
    # TODO: dW1 = ...
    # TODO: db1 = ... with shape (hidden,)

    return dW1, db1, dW2, db2

def update_parameters(W1, b1, W2, b2, gradients, learning_rate):
    dW1, db1, dW2, db2 = gradients
    # TODO: subtract learning_rate * gradient from every parameter
    return W1, b1, W2, b2
```

1. Run the function on Tuesday’s single example. Print all four gradients and compare them with Wednesday’s hand calculation.

2. Implement a finite-difference check for $W2[0, 1]$ using $\varepsilon = 10^{-5}$:

$$\frac{L(w + \varepsilon) - L(w - \varepsilon)}{2\varepsilon}.$$

Report the analytical gradient, numerical gradient, and absolute difference.

3. Repeat the check for $W1[1, 0]$ and $b1[1]$.

4. Explain why checking every parameter by finite differences would be far slower than backpropagation.

Debugging rule. If the gradient check fails, do not change the numerical-gradient formula until you have checked array shapes, transposes, averaging, the ReLU derivative, and whether the same unchanged parameters were used for both losses.

Friday: Train on a Tiny Nonlinear Dataset

Use this synthetic dataset. The positive examples lie mainly in the lower-left and upper-right regions, so one straight decision boundary is not sufficient.

```
X = np.array([
    [-1.2, -1.0], [-1.0, -0.5], [-0.6, -1.1],
    [ 1.1,  1.2], [ 0.6,  1.0], [ 1.2,  0.5],
```

```

    [-1.1, 1.0], [-0.6, 0.8], [-1.0, 0.3],
    [ 1.0, -1.1], [ 0.5, -0.9], [ 1.1, -0.4],
], dtype=float)

```

```

y = np.array([
    [1], [1], [1], [1], [1], [1],
    [0], [0], [0], [0], [0], [0]
], dtype=float)

```

```

rng = np.random.default_rng(15)
W1 = rng.normal(0, 0.6, size=(4, 2))
b1 = np.full(4, 0.1)
W2 = rng.normal(0, 0.6, size=(1, 4))
b2 = np.zeros(1)

```

Complete the training loop.

```

learning_rate = 0.1
epochs = 3000
loss_history = []

for epoch in range(epochs):
    # TODO: forward pass
    # TODO: calculate and store loss
    # TODO: backward pass
    # TODO: update all parameters

    if epoch % 300 == 0:
        # TODO: classify at threshold 0.5 and print loss and accuracy
        pass

```

1. Plot loss against epoch. Label both axes and give the figure a meaningful title.
2. Report the final probabilities, predicted labels, confusion matrix, accuracy, precision, recall, and F1.
3. Run again with learning rates .001, .01, .1, and 1.0. Create a table containing initial loss, final loss, and final accuracy. Explain the differences.
4. Change the hidden-layer width from 4 to 1, 2, and 8 neurons. Which versions learn the pattern reliably with this initialization? Explain why more neurons do not guarantee a better result.
5. Explain why performance on these same 12 training points is not evidence of performance on future data.
6. Optional: create a grid of (x_1, x_2) locations, predict each location, and draw the learned probability surface or decision regions behind the data points.

GPU Acceleration and Memory Explanation

Create one original diagram using draw.io or excalidraw with two panels.

1. **CPU versus GPU panel:** show a matrix multiplication $Z = XW^T$ whose many output cells can be calculated in parallel. Use the analogy “a few versatile workers” versus “many arithmetic workers,” but also state that a GPU has setup and data-transfer overhead.

2. **Inference versus training panel:** place these items in the correct side(s): model weights, input batch, current activations, saved forward activations, parameter gradients, optimizer state, and temporary workspace.

Answer the following calculations and questions.

1. A model has 10 million parameters stored as 32-bit floats. Using 4 bytes per value, approximately how many decimal MB do the weights occupy?
2. During training, suppose we store weights, one gradient per parameter, and two Adam-like optimizer values per parameter, all in 32-bit floats. Estimate the parameter-related subtotal before activations and temporary workspaces.
3. Explain why training must preserve many forward activations until their backward calculations are complete, while inference can often release earlier activations.
4. Name three ways to reduce training memory and one tradeoff for each.
5. Explain why a tiny one-example network may not run faster on a GPU even though a large mini-batch does.

Feynman Technique Post

Write a beginner-friendly learning note titled “How a Neural Network Learns from One Mistake.” It must:

1. draw or describe a two-input, two-hidden-neuron, one-output MLP;
2. distinguish prediction, loss, and an evaluation metric;
3. show one forward calculation and one binary cross-entropy value;
4. explain a gradient as a local direction signal;
5. trace the chain rule from loss to an early-layer weight;
6. show one gradient-descent update and its effect on loss;
7. explain GPU parallelism with a matrix example; and
8. explain why training generally uses more GPU memory than inference.

Submission checklist. Metric calculations and threshold table; labeled forward-pass network; complete hand-derived backward pass; matching NumPy gradients; three finite-difference checks; training loop and loss plot; architecture and learning-rate comparison; GPU/memory diagram and calculations; Feynman post; one sentence disclosing any permitted AI help.